NFL BIG DATA BOWL 2026 - PREDICTION

Predição do Movimento Vetorial de Jogadores

Diego Hiroshi Goto Yanaze - 626457
Guilherme Lorenzetti Bonini - 617547
Diogo Tachibana de Oliveira - 626872
Luciano Moreira Barbosa Junior - 625418
Luís Fernando - 629881

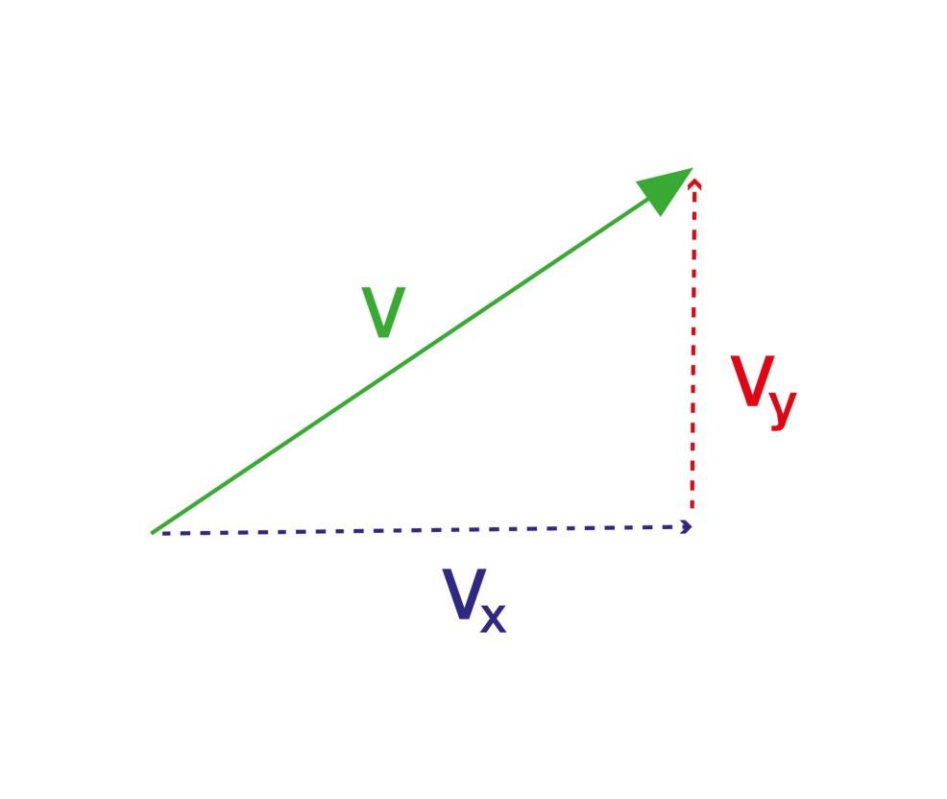
O PROBLEMA: REGRESSÃO MULTI-SAÍDA

- 🏈 **Objetivo:** Prever o movimento dos jogadores (velocidade futura) enquanto a bola está no ar.
- **Alvo (Y):** Previsão de duas variáveis contínuas: as componentes de velocidade **V_x** (horizontal) e **V_y** (vertical).
- **Métrica: RMSE** (Root Mean Squared Error).

ANÁLISE EXPLORATÓRIA DE DADOS (EDA) E LIMPEZA

- **Dataset:** Utilizado o arquivo `train/input_2023_wXX.csv` (várias semanas combinadas).
- **Limpeza:** O conjunto de dados se mostrou **extremamente limpo**, com **zero valores faltantes** (NaN) nas *features* críticas de movimento.
- **Variáveis Chave:** Posição do jogador, `x/y` (coordenadas), `s` (velocidade), `dir`/`o` (direção/orientação).

FEATURE ENGINEERING: CRIANDO O VETOR DE MOVIMENTO



PRÉ-PROCESSAMENTO: CODIFICAÇÃO E ESCALONAMENTO

- **One-Hot Encoding:** Aplicado a variáveis categóricas como `play_direction`, `player_position` e `player_role` para convertê-las em formato numérico binário.
- **Conversão de Altura:** A coluna `player_height` (ex: "6-1") foi convertida para polegadas totais, tornando-a numérica.
- **StandardScaler (Padronização):** Essencial para colocar todas as *features* (coordenadas x, y, peso, aceleração) na mesma escala (u=0, sigma=1), impedindo que grandes números dominem o treinamento.

MODELAGEM: XGBOOST REGRESSOR

[16]
27s

```python
import xgboost as xgb

# Inicializa o modelo (Regressor para prever valores contínuos)
# O XGBoost lida bem com Multi-Output Regression
model = xgb.XGBRegressor(objective='reg:squarederror', n_estimators=500, learning_rate=0.05, n_jobs=-1, random_state=42)

# Treina o modelo
model.fit(X_train, Y_train)
```

```
XGBRegressor
XGBRegressor(base_score=None, booster=None, callbacks=None,
             colsample_bylevel=None, colsample_bynode=None,
             colsample_bytree=None, device=None, early_stopping_rounds=None,
             enable_categorical=False, eval_metric=None, feature_types=None,
             feature_weights=None, gamma=None, grow_policy=None,
             importance_type=None, interaction_constraints=None,
             learning_rate=0.05, max_bin=None, max_cat_threshold=None,
             max_cat_to_onehot=None, max_delta_step=None, max_depth=None,
             max_leaves=None, min_child_weight=None, missing=nan,
             monotone_constraints=None, multi_strategy=None, n_estimators=500,
             n_jobs=-1, num_parallel_tree=None, ...)
```

O XGBoost é um método de *Gradient Boosting* baseado em árvores de decisão. É reconhecido por sua alta **eficiência**, **velocidade** e por ser ideal para problemas de **Regressão Multi-Saída** em dados tabulares.

RESULTADOS E MÉTRICA (RMSE)

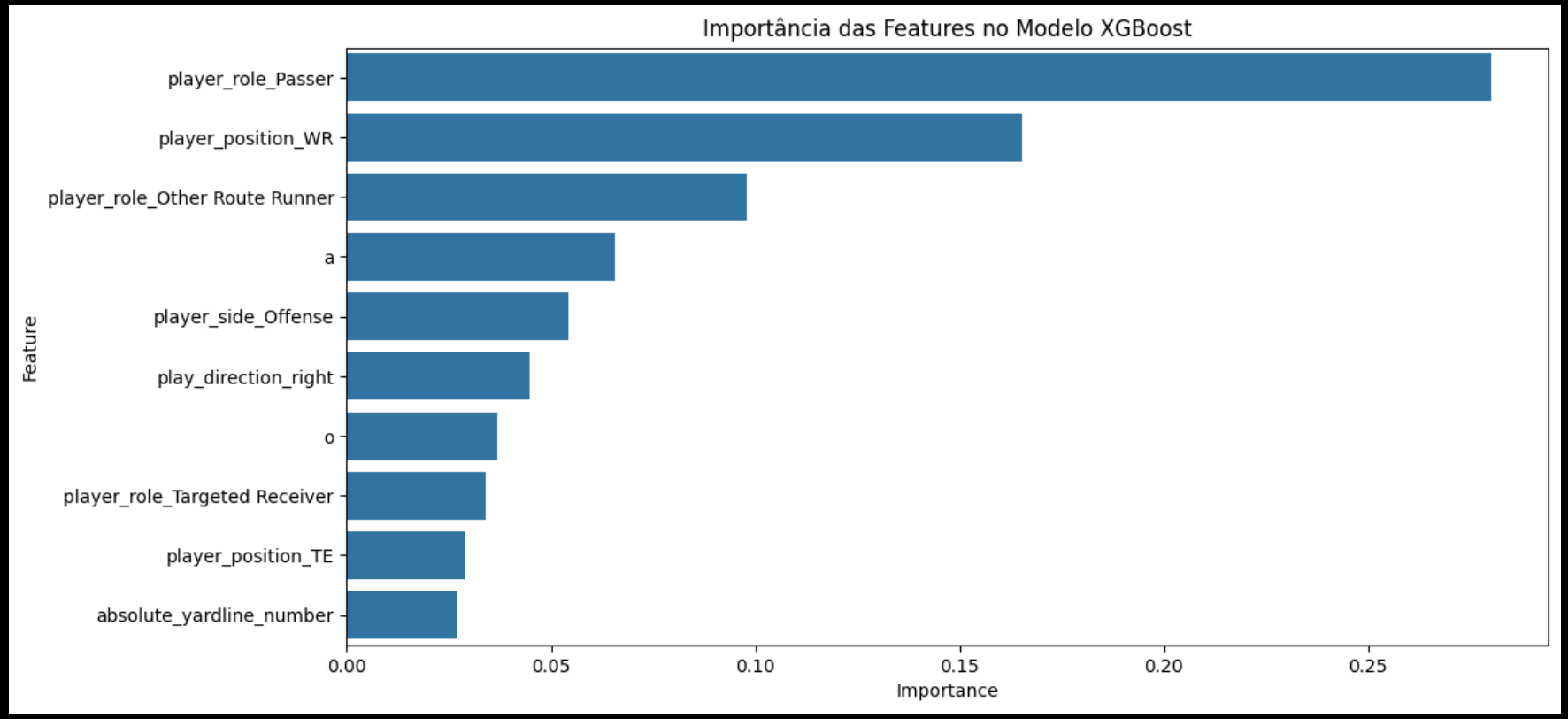
[23]
0s

```python
# 1. Calcular o Erro Quadrático Médio (MSE) para cada saída
mse_vx = mean_squared_error(Y_val['Vx'], Y_pred[:, 0]) # Y_val['Vx'] vs. a primeira coluna de Y_pred
mse_vy = mean_squared_error(Y_val['Vy'], Y_pred[:, 1]) # Y_val['Vy'] vs. a segunda coluna de Y_pred

# 2. Calcular o RMSE para cada saída
rmse_vx = np.sqrt(mse_vx)
rmse_vy = np.sqrt(mse_vy)

# 3. Calcular o RMSE total (usando o RMSE médio das duas saídas)
rmse_total = (rmse_vx + rmse_vy) / 2

print(f"\nResultado da Avaliação do Modelo:")
print(f"RMSE para a Velocidade Horizontal (Vx): {rmse_vx:.4f}")
print(f"RMSE para a Velocidade Vertical (Vy): {rmse_vy:.4f}")
print(f"RMSE Médio Total: {rmse_total:.4f}")
```

```
Resultado da Avaliação do Modelo:
RMSE para a Velocidade Horizontal (Vx): 0.0127
RMSE para a Velocidade Vertical (Vy): 0.6110
RMSE Médio Total: 0.3119
```

O erro médio de 0.3119 unidades (jardas/segundo) é um resultado forte, mostrando que o modelo é capaz de prever as mudanças de velocidade dos jogadores com precisão, especialmente na direção horizontal V_x

ANÁLISE PREDITIVA: O QUE O MODELO APRENDEU



As *features* mais importantes não são as coordenadas, mas sim o **Papel do Jogador** (`player_role_Passer`, `player_role_WR`).**Conclusão:** O modelo aprendeu que a **função tática** de um jogador em uma jogada é o fator que mais determina a natureza do seu movimento futuro.

CONCLUSÃO

- ✔ **Problema Resolvido:** Implementação de uma solução completa de Regressão Multi-Saída.
- ✔ **Código Validado:** O Notebook foi submetido ao Kaggle para ser re-executado em um conjunto de teste oculto, comprovando a validade do código.

OBRIGADO À TODOS!

- Diego Hiroshi Goto Yanaze - 626457
- Guilherme Lorenzetti Bonini - 617547
- Diogo Tachibana de Oliveira - 626872
- Luciano Moreira Barbosa Junior - 625418
- Luís Fernando - 629881